

PROMTS

Prompt que inserte:

Actúa como asistente de programación para una práctica de Estructura de Datos en Python.

Contexto: Ya tengo un proyecto desarrollado en clase con Python, programación orientada a objetos y una interfaz gráfica creada con Qt Designer. En el proyecto ya existe una clase Stack con los métodos básicos de una pila. También existe una ventana o formulario donde se capturan datos y se muestran resultados, así como la clase que realiza la carga y la detección del evento para convertir la expression. Objetivo: Necesito agregar al proyecto existente la funcionalidad para convertir una expresión infija a expresión posfija utilizando la estructura de pila ya creada en clase. Necesito se genere una clase que contenga una pila y un método para llevar acabo la conversion.

Instrucciones importantes:

1. No generes un proyecto nuevo desde cero.
2. No reemplaces la clase Stack existente. Debes utilizarla e integrarte a su forma actual de trabajo.
3. Respeta la organización actual de archivos, clases, nombres de métodos y estilo de programación del proyecto.
4. Si necesitas proponer cambios, indícalos como cambios mínimos y explica exactamente en qué archivo y en qué parte deben colocarse.
5. La solución debe estar hecha con programación orientada a objetos.
6. La interfaz gráfica debe seguir usando el diseño generado con Qt Designer; no cambies el archivo .ui salvo que sea estrictamente necesario.
7. La aplicación debe permitir capturar una expresión infija, convertirla a posfija y mostrar el resultado en la interfaz.
8. Se asume que la expression infija capturada es válida ya que de otra forma la conversion puede fallar.
9. Usa operadores básicos: +, -, *, /, \$(Potencia) y paréntesis. 10. Explica brevemente cómo funciona el algoritmo y cómo se utiliza la pila en la conversion, para esto genera un archivo .md que contenga esta explicación.

Código:

```
class Stack:
    def __init__(self):
        self.top = None

    def is_empty(self):
        return self.top is None
```

Insertar un elemento en la pila

```
def push(self, data):
    new_node = Node(data)
    if self.top is None:
        self.top = new_node
    else:
        new_node.next = self.top
        self.top = new_node
```

Eliminar un elemento

#Retornar el valor eliminado

```
def pop(self): if self.top is None: return None valor = self.top.data self.top = self.top.next return valor
```

#Retornar el valor del nodo en el tope

```
def top_of_stack(self): if self.top != None: return self.top.data
```

Muestra el contenido de la pila

```
def print_stack(self):  
    temp = self.top  
    valores = []  
    while temp is not None:  
        valores.append(str(temp.data))  
        temp = temp.next  
    return "Top-> " + "->".join(valores)
```

```
import sys
```

```
from PyQt5.QtWidgets import QApplication
```

```
from loads.Id_menu_principal import VentanaPrincipal
```

```
def main():
```

```
    app = QApplication(sys.argv)
```

```
    menu = VentanaPrincipal()
```

```
    menu.show()
```

```
    sys.exit(app.exec_())
```

```
if name == "main":
```

```
    main() ,
```

```
from PyQt5 import uic
from PyQt5.QtWidgets import QDialog
from estructuras.lineales.pila_stack import Stack
from estructuras.lineales.nodo import Node
```

```
class Dg_Stack(QDialog):
```

```
    def init(self):
```

```
        super().init() uic.loadUi("ui/Pila_stack.ui", self)
```

```
        self.da = None
```

```
        self.ag = Stack()
```

```
        self.btn_agr.clicked.connect(self.agre)
```

```
        self.btn_elim.clicked.connect(self.elimi)
```

```
        self.btn_imp.clicked.connect(self.prin)
```

```
        self.btn_mos.clicked.connect(self.to)
```

```
    def agre(self):
```

```
        self.da = self.ent_dato.text().strip()
```

```
        if self.da:
```

```
            self.ag.push(self.da)
```

```
            self.ent_dato.clear()
```

```
        else:
```

```
            self.lbl_resultado.setText("Ingrese un valor")
```

```
    def elimi(self):
```

```
        eliminado = self.ag.pop()
```

```
        if eliminado is None:
```

```

        self.lbl_resultado.setText("Pila vacía")
    else:
        self.lbl_resultado.setText(f"Se eliminó: {eliminado}")

def prin(self):
    self.lbl_resultado.setText(self.ag.print_stack())

def to(self):
    self.lbl_resultado.setText(f"Top-> {self.ag.top_of_stack()}")

```

Resultado esperado de tu respuesta:

- Primero analiza la estructura del código existente.
- Después propón la clase con el método para convertir de infija a posfija.
- Luego indica cómo conectar ese método con el botón o evento de la interfaz gráfica.
- Finalmente proporciona pruebas para comprobar que funciona.
- No omitas explicación, porque debo entender el código para defenderlo en clase ante el profesor y mis compañeros.

Resultado que me dio:

1. Análisis del código existente

- Ya tienes una clase Stack implementada con nodos (Node) que permite:
 - push() insertar elementos.
 - pop() eliminar y devolver el tope.
 - top_of_stack() consultar el tope.
 - is_empty() verificar si está vacía.
- Existe una ventana (Dg_Stack) creada con Qt Designer que permite interactuar con la pila: agregar, eliminar, imprimir y mostrar el tope.
- El flujo principal (main) carga la ventana principal y ejecuta la aplicación.

👉 Esto significa que la pila ya está lista y funcionando. Lo único que falta es **crear una clase que use esa pila para convertir expresiones infijas a posfijas** y conectarla a la interfaz.

2. Clase para convertir de infija a posfija

Se propone crear un archivo nuevo en la misma carpeta de estructuras, por ejemplo: estructuras/lineales/infix_to_postfix.py

```
from estructuras.lineales.pila_stack import Stack
```

```
class InfixToPostfix:
```

```
    def __init__(self):
```

```
        self.stack = Stack()
```

```
        self.output = []
```

```
    def precedence(self, op):
```

```
        if op in ('+', '-')
```

```
            return 1
```

```
        if op in ('*', '/'):
```

```
            return 2
```

```
        if op == '$': # potencia
```

```
            return 3
```

```
        return 0
```

```
    def is_operator(self, c):
```

```
        return c in ['+', '-', '*', '/', '$']
```

```
    def convert(self, expression: str) -> str:
```

```
        self.output = [] # limpiar salida en cada conversión
```

```
        for token in expression:
```

```
            if token.isalnum(): # operando
```

```

        self.output.append(token)

    elif token == '(':
        self.stack.push(token)

    elif token == ')':
        while not self.stack.is_empty() and self.stack.top_of_stack() != '(':
            self.output.append(self.stack.pop())
        self.stack.pop() # eliminar '('

    elif self.is_operator(token):
        while (not self.stack.is_empty() and
               self.precedence(self.stack.top_of_stack()) >= self.precedence(token)):
            self.output.append(self.stack.pop())
        self.stack.push(token)

    while not self.stack.is_empty():
        self.output.append(self.stack.pop())

    return "".join(self.output)

```

3. Conexión con la interfaz gráfica

En tu clase Dg_Stack agrega:

python

```
from estructuras.lineales.infix_to_postfix import InfixToPostfix
```

Y un nuevo método:

python

```

def convertir(self):
    expresion = self.ent_dato.text().strip()

    if expresion:
        convertidor = InfixToPostfix()
        resultado = convertidor.convert(expresion)

```

```
self.lbl_resultado.setText(f"Posfija: {resultado}")
```

```
else:
```

```
self.lbl_resultado.setText("Ingrese una expresión")
```

En el `__init__` conecta el botón que hayas definido en Qt Designer (ejemplo `btn_conv`):

python

```
self.btn_conv.clicked.connect(self.convertir)
```

👉 Con esto, al escribir una expresión en el `QLineEdit` y presionar el botón “Convertir”, se mostrará la expresión posfija en el `QLabel`.

4. Pruebas para comprobar que funciona

Ejemplo de entrada y salida:

- Entrada:

Código

$A*(B+C)/D$

- Proceso interno:
 - $A \rightarrow$ salida
 - $*$ $\rightarrow$ pila
 - $(\rightarrow$ pila
 - $B \rightarrow$ salida
 - $+ \rightarrow$ pila
 - $C \rightarrow$ salida
 - $) \rightarrow$ desapilar hasta $($
 - $/ \rightarrow$ pila
 - $D \rightarrow$ salida

- Salida esperada:

Código

$ABC+*D/$

Otro ejemplo:

- Entrada: $A+B*C$
- Salida: $ABC*+$

5. Explicación para defenderlo en clase

El algoritmo usado es el **Shunting Yard de Dijkstra**:

- La pila se utiliza para **guardar operadores y paréntesis** mientras se recorre la expresión.
- Los operandos se envían directamente a la salida.
- Los operadores se comparan con los que están en la pila según su **precedencia**:
 - Si el operador en la pila tiene mayor o igual precedencia, se desapila y se manda a la salida.
 - Luego se apila el nuevo operador.
- Los paréntesis controlan el orden:
 - (se apila.
 -) desapila hasta encontrar (.
- Al final, se desapilan todos los operadores restantes.

Esto garantiza que la expresión posfija respete el orden correcto de evaluación sin necesidad de paréntesis.

Dudas que tuve:

para que es esta funcion y como se ejecuta :

```
def precedence(self, op):
```

```
    if op in ('+', '-')
```

```
        return 1
```

```
    if op in ('*', '/')
```

```
        return 2
```

```
    if op == '^': # potencia
```

```
        return 3
```

```
    return 0
```

RESPUESTA:

Esta función **define la precedencia de los operadores matemáticos**. En el algoritmo de conversión de infija a posfija, necesitamos saber qué operador “pesa más” para decidir si se apila o se desapila.

- + y - → precedencia **1** (más baja).
- * y / → precedencia **2** (más alta que suma/resta).
- \$ (potencia) → precedencia **3** (la más alta).
- Cualquier otro símbolo → precedencia **0** (no es operador reconocido).

👉 Gracias a esta función, el algoritmo puede comparar operadores en la pila y decidir si debe desapilarlos antes de apilar el nuevo.

⚙️ ¿Cómo se ejecuta?

No se ejecuta sola, sino que se **llama dentro del método** `convert()` cuando se procesa un operador. Ejemplo de uso directo:

python

```
conv = InfixToPostfix()
print(conv.precedence('+')) # 1
print(conv.precedence('*')) # 2
print(conv.precedence('$')) # 3
print(conv.precedence('A')) # 0 (no es operador)
```

🔴 Salida:

Código

```
1
2
3
0
```

En el flujo completo de conversión, se usa así:

python

```
elif self.is_operator(token):
    while (not self.stack.is_empty() and
           self.precedence(self.stack.top_of_stack()) >= self.precedence(token)):
```

```
self.output.append(self.stack.pop())
```

```
self.stack.push(token)
```

Aquí la función `precedence()` permite comparar el operador actual con el que está en el tope de la pila y decidir si se desapila o se mantiene.

👉 En resumen:

- **La función sirve para asignar un nivel de prioridad a cada operador.**
- **Se ejecuta automáticamente dentro del algoritmo de conversión cuando se evalúan operadores.**

¿Quieres que te muestre un ejemplo paso a paso de cómo se usa esta función en la conversión de una expresión concreta, como $A+B*C$?